PROG5 (GBIN5U03) - Examen de seconde session
Année 2024-2025

Durée : 2h.
Documents autorisés: Aucun, à l'exception d'un dictionnaire bilingue non annoté pour les étudiants étrangers. Tout appareil electronique interdit. Pour chaque question, une partie des points peut tenir compte de la présentation.
Le barême est indicatif.

UFR IM²AG

UNIVERSITÉ Grenoble Alpes

1 Histoires de matrices (1 point)

Dans ce sujet nous allons écrire un ensemble de fonctions de manipulation de matrices d'entiers. Vous disposez d'un fichier `matrice.h` dont le contenu vous est donné ci-dessous. Dans un premier temps, vous devez écrire le fichier `matrice.c` associé.

```
typedef struct donnees_matrice {
  int l,c;
  int **donnees;
} *matrice;

/* une matrice telle que est_matrice_invalide(matrice_invalide) retourne 1 */
extern matrice matrice_invalide;

/* description : teste si une matrice est invalide
 * parametres : la matrice
 * retour : 1 si la matrice est invalide, 0 sinon
 * effets de bord : aucun */
int est_matrice_invalide(matrice m);

/* description : retourne un pointeur vers un element de matrice
 * parametres : la matrice et les indices de l'élément
 * retour : un pointeur vers int
 * effets de bord : aucun */
int *acces_matrice(matrice m, int i, int j);

/* description : nombre de lignes d'une matrice
 * parametres : la matrice
 * retour : son nombre de lignes
 * effets de bord : aucun */
int nb_lignes_matrice(matrice m);

/* description : nombre de colonnes d'une matrice
 * parametres : la matrice
 * retour : son nombre de colonnes
 * effets de bord : aucun */
int nb_colonnes_matrice(matrice m);
```

2 Allocation mémoire (5 points)

Ecrivez les fonctions :

- `allouer_matrice` prenant en paramètres les dimensions d'une matrice et retournant une matrice du type décrit en introduction, pour laquelle de la mémoire aura été allouée de manière à pouvoir stocker tous ses éléments. Une matrice invalide devra être retournée si l'allocation échoue ;
- `liberer_matrice` permettant de libérer la mémoire allouée à une matrice ;
- `redimensionner_matrice` prenant en paramètres une matrice et de nouvelles dimensions. Les dimensions de la matrice donnée devront être modifiées pour devenir les nouvelles dimensions. Si le redimensionnement de la matrice échoue (typiquement lorsque le redimensionnement des zones mémoire qui lui sont allouées échoue), la matrice devra devenir une matrice invalide. Attention, le redimensionnement doit pouvoir se faire dans tous les sens (les nombres de lignes et de colonnes doivent pouvoir tous les deux soit augmenter, soit diminuer).

Attention, ces fonctions ne devront créer aucune fuite mémoire.

3 Entrées/Sorties (3 points)

Ecrivez les fonctions :

- `ecrire_matrice` qui devra écrire sous forme textuelle (humainement lisible) la matrice donnée en second paramètre sur le flux de sortie donné en premier paramètre ;
- `lire_matrice` qui devra lire une matrice au même format que celui produit par `ecrire_matrice`, depuis un flux d'entrée donné en paramètre et devra retourner la matrice lue.

Le format textuel choisi pour la matrice est laissé libre mais :

- il doit être le même pour les deux fonctions ;
- vous devez expliquer comment il est organisé ;
- vous devez donner un exemple pour illustrer votre explication.

4 Opérations (3 points)

Ecrivez deux fonctions `somme_matrices` et `produit_matrices`, prenant toutes les deux trois matrices, `a`, `b` et `c` en paramètre, qui devront :

- redimensionner `c` pour pouvoir accueillir le résultat de l'opération demandée
- calculer dans `c` le résultat :
  - de la somme de `a` et `b` pour la fonction `somme_matrices`
  - du produit de `a` par `b` pour la fonction `produit_matrices`

Cette fonction devra retourner 0 en cas d'erreur (redimensionnement, matrices de taille incompatibles) et 1 en cas de succès.

5 Programme (4 points)

Ecrivez un programme prenant en arguments de la ligne de commande un nom d'opération et deux noms de fichiers. Ce programme devra afficher le resultat de l'application de l'opération donnée aux matrices stockées dans les fichiers de nom donné. Les opérations à implémenter seront :

- la somme ;
- le produit.

Votre programme devra utiliser les fonctions écrites en section 4 pour effectuer ce calcul. Votre programme devra également afficher un message et se terminer en cas d'erreur (quelque soit l'erreur).

6 Makefile (2 points)

En supposant que les réponses aux questions précédentes soient placées dans un fichier `.c` séparé pour chacune des questions, décrivez l'ensemble des fichiers formant l'application complète (programme de la question 5) et écrivez un `Makefile` permettant de compiler cette application sans utiliser de règle implicite. Votre `Makefile` devra compiler l'application de manière séparée et la fabrication des fichiers objet devra être décrite avec une règle générique.

7 Allocation mémoire, le retour (2 points)

Reprendre les fonctions de la question 2 en n'utilisant qu'un seul malloc pour `allouer_matrice` et qu'un seul free pour `liberer_matrice` (et sans changer le code fourni).